Ejercicio práctico : Normalización 1FN → 2FN → 3FN en PostgreSQL

Objetivo

Tomar una tabla “plana” con listas en una celda y **normalizarla gradualmente** hasta 3FN, entendiendo el **problema que resuelve cada forma normal**.

Escenario inicial (tabla no normalizada)

En una sola tabla guardamos: datos del estudiante + **listas** de cursos e instructores en **una celda** (mal diseño).

```
-- Limpieza por si existía antes

DROP TABLE IF EXISTS enrollments_raw;



CREATE TABLE enrollments_raw (

  student_id         INT,

  student_name       TEXT,

  student_email      TEXT,

  course_ids         TEXT, -- p.ej. 'MAT101|PHY101'

  course_names       TEXT, -- p.ej. 'Álgebra|Física'

  instr_names        TEXT, -- p.ej. 'Dra. Rojas|Dr. Vidal'

  instr_offices      TEXT  -- p.ej. 'B-201|A-105'

);



INSERT INTO enrollments_raw VALUES

(1, 'Ana Pérez',  'ana@uni.cl',  'MAT101|PHY101',  'Álgebra|Física', 'Dra. Rojas|Dr. Vidal', 'B-201|A-105'),

(2, 'Luis Soto',  'luis@uni.cl', 'MAT101|HIS100',  'Álgebra|Historia','Dra. Rojas|Mg. León', 'B-201|C-310');
```

Qué está mal y por qué normalizar

- **No cumple 1FN**: hay **grupos repetidos** (listas separadas por "|").
- Se repiten nombres/atributos en varias filas → **riesgo de inconsistencia**.
- Es difícil consultar (contar alumnos por curso, etc.) y mantener.

Paso 1 — Primera Forma Normal (1FN)

Meta: hacer cada celda **atómica** (un solo valor) y cada fila una **ocurrencia** clara.
Idea: separar las listas en filas individuales (una fila por estudiante–curso).

Diseño 1FN (simple)

- `enrollments_1fn(student_id, student_name, student_email, course_id, course_name, instr_name, instr_office)`

  Aún está "cargada" (mezcla datos de alumno, curso e instructor), pero **ya no tiene listas**.

SQL 1FN

```
DROP TABLE IF EXISTS enrollments_1fn;


CREATE TABLE enrollments_1fn (

  student_id    INT NOT NULL,

  student_name  TEXT NOT NULL,

  student_email TEXT NOT NULL,

  course_id     TEXT NOT NULL,

  course_name   TEXT NOT NULL,

  instr_name    TEXT NOT NULL,

  instr_office  TEXT NOT NULL

);
```

```
-- Dividimos las listas con regexp_split_to_table + WITH ORDINALITY para
alinear posiciones

WITH base AS (

  SELECT

    student_id, student_name, student_email,

    regexp_split_to_table(course_ids,   '\|') WITH ORDINALITY AS
c_id(course_id, n1),

    regexp_split_to_table(course_names, '\|') WITH ORDINALITY AS
c_nm(course_name, n2),

    regexp_split_to_table(instr_names,  '\|') WITH ORDINALITY AS
i_nm(instr_name, n3),

    regexp_split_to_table(instr_offices,'\|') WITH ORDINALITY AS
i_of(instr_office, n4)

  FROM enrollments_raw

),

aligned AS (

  SELECT

    student_id, student_name, student_email,

    c_id.course_id, c_nm.course_name, i_nm.instr_name, i_of.instr_office

  FROM base

  WHERE c_id.n1 = c_nm.n2 AND c_id.n1 = i_nm.n3 AND c_id.n1 = i_of.n4

)

INSERT INTO enrollments_1fn

SELECT * FROM aligned;



-- Verifica

SELECT * FROM enrollments_1fn ORDER BY student_id, course_id;
```

Qué hiciste (cómo/por qué/para qué)

- **Cómo:** partiste las listas por "|" y alineaste por posición para reconstruir pares estudiante–curso–instructor.
- **Por qué:** garantizar **celdas atómicas** (1FN) y permitir **consultas** limpias.
- **Para qué:** ahora puedes contar estudiantes por curso, filtrar por instructor, etc., sin "parsear" strings.

Diagrama (1FN)

```
erDiagram
    ENROLLMENTS_1FN {
      int  student_id
      text student_name
      text student_email
      text course_id
      text course_name
      text instr_name
      text instr_office
    }
```

Paso 2 — Segunda Forma Normal (2FN)

Meta: eliminar **dependencias parciales** sobre una **clave compuesta**.
Si consideras la clave de `enrollments_2fn` como `(student_id, course_id)`, observa:

- `student_name` y `student_email` **dependen solo de student_id**.
- `course_name`, `instr_name`, `instr_office` **dependen solo de course_id**.
  Eso **viola 2FN**. Solución: **separar** entidades.

Diseño 2FN (simple)

- `students(student_id, student_name, student_email)`

- `courses(course_id, course_name, instr_name, instr_office)` *(aún incluye oficina del instructor)*
- `enrollments(student_id, course_id)` **(PK compuesta)**

SQL 2FN

```
DROP TABLE IF EXISTS enrollments CASCADE;

DROP TABLE IF EXISTS courses CASCADE;

DROP TABLE IF EXISTS students CASCADE;


CREATE TABLE students (

  student_id    INT PRIMARY KEY,

  student_name  TEXT NOT NULL,

  student_email TEXT NOT NULL UNIQUE

);


CREATE TABLE courses (

  course_id     TEXT PRIMARY KEY,

  course_name   TEXT NOT NULL,

  instr_name    TEXT NOT NULL,

  instr_office  TEXT NOT NULL

);


CREATE TABLE enrollments (

  student_id INT NOT NULL REFERENCES students(student_id) ON DELETE
CASCADE,

  course_id  TEXT NOT NULL REFERENCES courses(course_id)  ON DELETE
CASCADE,

  PRIMARY KEY (student_id, course_id)
```

```
);


-- Poblar desde la tabla 1FN

INSERT INTO students(student_id, student_name, student_email)

SELECT DISTINCT student_id, student_name, student_email

FROM enrollments_1fn;


INSERT INTO courses(course_id, course_name, instr_name, instr_office)

SELECT DISTINCT course_id, course_name, instr_name, instr_office

FROM enrollments_1fn;


INSERT INTO enrollments(student_id, course_id)

SELECT DISTINCT student_id, course_id

FROM enrollments_1fn;


-- Verifica

SELECT * FROM students ORDER BY student_id;

SELECT * FROM courses  ORDER BY course_id;

SELECT * FROM enrollments ORDER BY student_id, course_id;
```

Qué hiciste (cómo/por qué/para qué)

- **Cómo:** separaste **alumnos** y **cursos**; dejaste `enrollments` como **relación** M:N.
- **Por qué:** evitar que atributos dependan de **solo una parte** de la clave compuesta (2FN).
- **Para qué:** reducir **redundancia** y **errores de actualización** (si cambia el email del alumno, se actualiza en un solo lugar).

Diagrama (2FN)

```
erDiagram
    STUDENTS {
      int  student_id PK
      text student_name
      text student_email
    }
    COURSES {
      text course_id PK
      text course_name
      text instr_name
      text instr_office
    }
    ENROLLMENTS {
      int  student_id FK
      text course_id  FK
    }
    STUDENTS ||--o{ ENROLLMENTS : cursa
    COURSES  ||--o{ ENROLLMENTS : es_cursado_en
```

Paso 3 — Tercera Forma Normal (3FN)

Meta: eliminar **dependencias transitivas** (atributos no clave que dependen de otros no clave).
Aquí, en `courses`, el atributo `instr_office` **depende de** `instr_name` (no de `course_id` directamente): **transitiva**.
Solución: separar **instructores**.

Diseño 3FN (simple)

- `instructors(instr_name PK, instr_office)`

- courses(course_id, course_name, instr_name FK) *(sin office)*

SQL 3FN

```
-- Creamos instructores y movemos la oficina desde courses
DROP TABLE IF EXISTS instructors CASCADE;

CREATE TABLE instructors (
  instr_name   TEXT PRIMARY KEY,
  instr_office TEXT NOT NULL
);

-- Poblar instructores desde courses
INSERT INTO instructors(instr_name, instr_office)
SELECT DISTINCT instr_name, instr_office
FROM courses;

-- Ajustar courses: quitar instr_office y referenciar al instructor
ALTER TABLE courses DROP COLUMN instr_office;
ALTER TABLE courses
  ADD CONSTRAINT fk_courses_instructor
  FOREIGN KEY (instr_name) REFERENCES instructors(instr_name);

-- Verifica
SELECT * FROM instructors ORDER BY instr_name;
SELECT course_id, course_name, instr_name FROM courses ORDER BY course_id;
```

Qué hiciste (cómo/por qué/para qué)

- **Cómo:** creaste la tabla **instructors** y la referenciastes desde `courses`.
- **Por qué:** evitar **dependencias transitivas** (3FN). La oficina depende del instructor, no del curso.
- **Para qué:** mantener la **unicidad** y **consistencia** de los datos del instructor; si cambia la oficina, actualizas **una sola tabla**.

Diagrama (3FN)

```
erDiagram
    STUDENTS {
      int  student_id PK
      text student_name
      text student_email
    }
    INSTRUCTORS {
      text instr_name PK
      text instr_office
    }
    COURSES {
      text course_id PK
      text course_name
      text instr_name FK
    }
    ENROLLMENTS {
      int  student_id FK
      text course_id  FK
    }
```

```
    INSTRUCTORS ||--o{ COURSES     : imparte

    STUDENTS   ||--o{ ENROLLMENTS  : cursa

    COURSES    ||--o{ ENROLLMENTS  : es_cursado_en
```

Consultas útiles de comprobación

```
-- Alumnos con sus cursos e instructores (vía 3FN):

SELECT s.student_name, s.student_email, c.course_id, c.course_name,
i.instr_name, i.instr_office

FROM enrollments e

JOIN students   s ON s.student_id = e.student_id

JOIN courses    c ON c.course_id  = e.course_id

JOIN instructors i ON i.instr_name = c.instr_name

ORDER BY s.student_name, c.course_id;


-- ¿Cuántos alumnos por curso?

SELECT c.course_id, c.course_name, COUNT(*) AS alumnos

FROM enrollments e

JOIN courses c ON c.course_id = e.course_id

GROUP BY c.course_id, c.course_name

ORDER BY c.course_id;
```

Qué validas

- Integridad de relaciones (FKs).
- Que la información está **desnormalizada correctamente** (consultas sencillas, sin string-splitting).
- Métricas (alumnos por curso) salen **fácil**.

Resumen (qué resuelve cada FN)

- **1FN**: te obliga a **celdas atómicas** (sin listas en una celda) → facilita **consultar** y **validar** datos.
- **2FN**: elimina que atributos dependan de **parte** de una **clave compuesta** → **menos duplicación** y actualizaciones coherentes.
- **3FN**: elimina **transitivas** entre **no claves** → separa datos "maestros" (instructores) de datos "de relación" (cursos), reforzando la **consistencia**.